Portfolio Lab: Advanced Stock Trading and Portfolio Management Application

HCLTech Career Shaper | Project 25 | Individual Academic Project

Submitted by: Deepanshu Kaushik | Roll No.: 2300321540069

Validated dataset snapshot: 26 Oct 2021 – 26 Oct 2026 | Report generated: 26 Oct 2026

> **Simulation only. Not investment advice.** Virtual money; no real brokerage connection.

Abstract

Portfolio Lab is a working virtual trading and analysis application covering eight NSE-listed stocks. It brings together an auditable SQLite trade ledger, a data-cleaning pipeline, cached market history, portfolio profit-and-loss (P&L) accounting, six risk measures, technical signals, a next-open backtest and a time-ordered classification model. The interface is a five-page Streamlit application with CSV exports and a simulation warning on every page.

The supplied source dataset contains 500 synthetic rows; cleaning repaired 126 invalid high bounds and 159 invalid low bounds without dropping any row. The active downloaded cache holds 11,123 validated stock and benchmark rows. The machine-learning evaluation is always reported next to a majority-class baseline and is never presented as investment performance.

Key results at a glance

Area	Result
Source data cleaned	500 rows kept; 126 high and 159 low bounds repaired; 0 rows dropped
Active market history	11,123 rows (8 stocks + NIFTY 50) from yfinance
Accounting check	Hand-verified example reproduced exactly (total P&L INR 160)
TCS SMA backtest	Strategy −23.94% vs buy-and-hold −26.96%
TCS classifier	Accuracy 52.74% vs majority baseline 53.59%: no evidence of an edge

1. Problem Statement and Objectives

A learner or portfolio analyst needs a safe place to see how price data, trade accounting, risk measurement and strategy testing fit together. A raw OHLCV table cannot, by itself, say how much virtual cash remains after fees, how a sale changes realised profit, or whether a trading signal would have survived realistic order timing. Portfolio Lab answers these questions with reproducible calculations and a local database, and never places a real order.

Measurable objectives

Objective	Acceptance evidence
Clean the supplied data	500 rows preserved; invalid candle bounds repaired
Maintain a virtual portfolio	Atomic orders, weighted-average cost, cash and share guards
Quantify risk	Six historical measures, with NIFTY 50 as benchmark where available
Evaluate signals honestly	Next-open fills, fees and a buy-and-hold comparison
Demonstrate ML discipline	Chronological holdout, label gap, baseline and confusion matrix

Objective	Acceptance evidence
Deploy reproducibly	Pinned dependencies, Docker Compose and a health endpoint

Project boundaries

The application uses virtual money only. It has no broker integration, no real-money execution, no authentication and no personal-data collection. Cached yfinance prices are historical observations, not live quotes. Every page shows a simulation disclaimer. All results are descriptive and do not justify a recommendation to trade.

How the five views answer the problem

- **Dashboard:** what the account is worth and why it changed.
- **Trade:** an auditable cash-and-share ledger.
- **Risk:** turns a price series into standard measures that can be compared with the NIFTY 50 benchmark.
- **Signals:** separates a visual indicator from its next-open execution rule.
- **Data Explorer:** exposes the rows and quality decisions behind every other view.

All pages read the same SQLite database, so state stays consistent across navigation and restarts.

2. Data Provenance and Quality

The original `dataset.csv` contains eight ticker symbols with date, OHLC prices, volume and a provided daily return. Its dates cover 63 consecutive calendar days in early 2024, so weekend rows reveal its synthetic nature. The original files are preserved untouched. The pipeline writes `cleaned_dataset.csv`, recalculates returns per ticker and records every change in `quality_report.json`.

Quality check	Observed / action
Rows / duplicate keys	500 / 0
Invalid high bounds	126 repaired using max(high, open, close)
Invalid low bounds	159 repaired using min(low, open, close)
Weekend rows	140 retained and labelled synthetic
Invalid rows dropped	0

Quality check	Observed / action
Active history	11,123 rows from yfinance

Real market cache

When available, yfinance supplies five years of `.NS` history for the eight stocks and `^NSEI` for the benchmark. The cache is activated only if each stock and NIFTY 50 has at least 252 rows. It is stored separately from the synthetic CSV. If the download or completeness check fails, the supplied data remains a working fallback, and annualised measures and ML are withheld rather than computed on inadequate data.

Cleaning rules

The cleaning function is vectorised with pandas. It:

1. parses dates and coerces numeric fields;
2. drops invalid identifiers and non-positive prices;
3. de-duplicates `(date, ticker)` keys;
4. expands candle bounds to include open and close;
5. sorts by ticker and date, then recomputes returns.

Each ticker's first close-to-close return is undefined. Volume must be non-negative and OHLC values positive; SQLite additionally enforces candle order and uniqueness of `(date, ticker)`. The quality report counts every action, so repairs are observable rather than silent. No synthetic row is ever appended to a downloaded price series.

3. Exploratory Analysis: Prices and Returns

Price levels differ substantially between companies and shift over time, so direct comparison of levels is descriptive, not risk-adjusted. The first figure shows closing prices for three stocks (TCS, RELIANCE, INFY) from the active downloaded cache. The second shows the recomputed daily returns of the supplied synthetic data. Because that sample is short and uses an artificial calendar, it should not be read as a market return distribution.

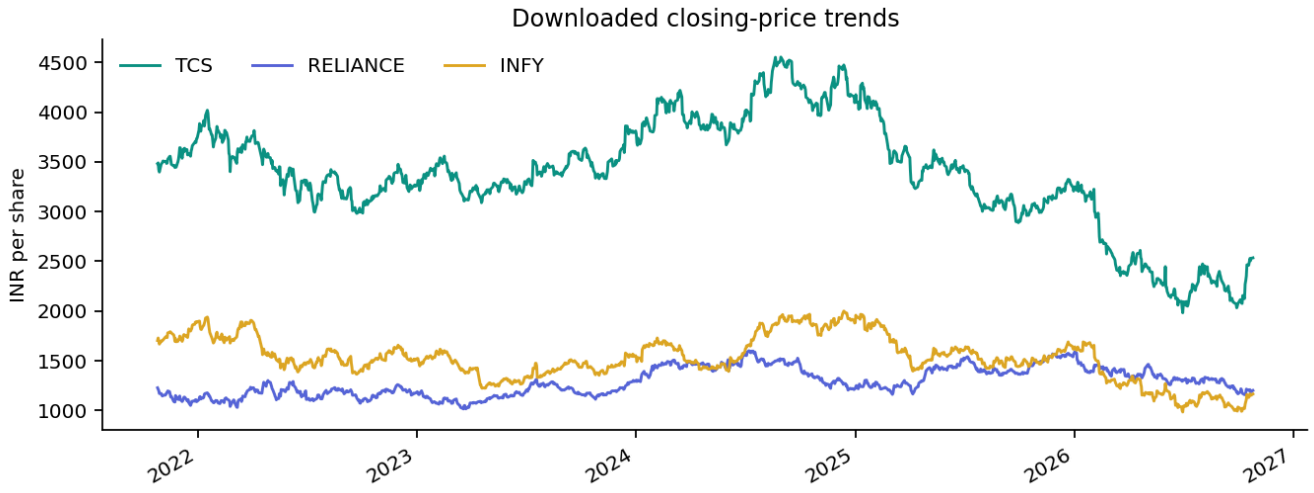


Figure 1: Downloaded closing-price trends, TCS / RELIANCE / INFY, 2022–2027

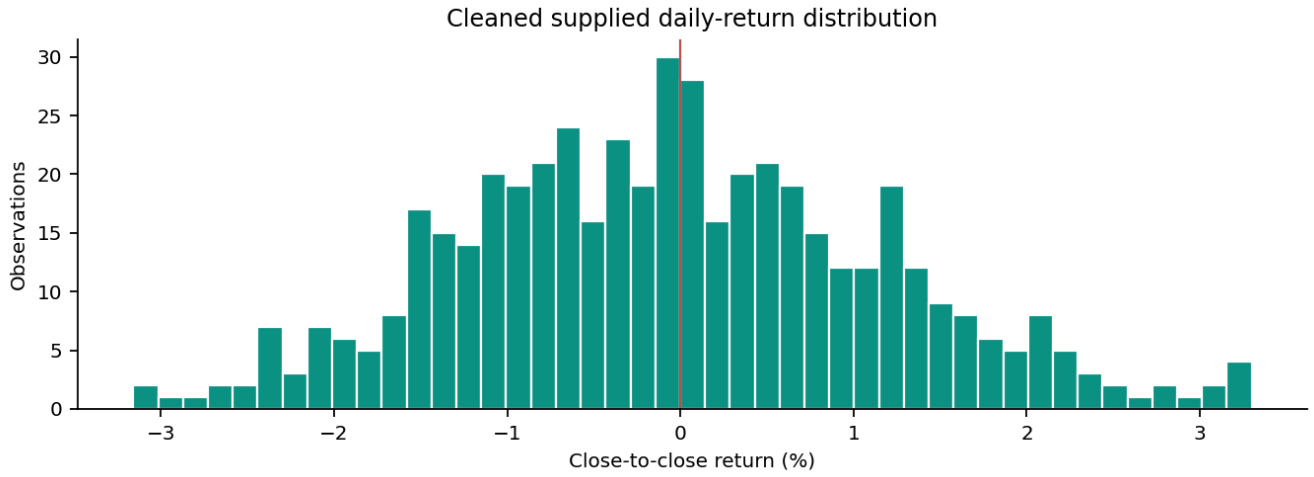


Figure 2: Cleaned supplied daily-return distribution (close-to-close return, %)

The executable notebook also contains a correlation heatmap and a volume comparison. The two data sources are kept apart on purpose: the repaired supplied data serves the academic data-cleaning exercise, while the longer real cache supports annualised calculations.

4. Exploratory Analysis: Co-movement and Volume

Pairwise correlations use only observations shared by two tickers' return series. They describe this sample alone and can change with the date window, missing data and market regime. Volume distributions show that tickers in the supplied dataset trade at different activity scales, which is why volumes are inspected per ticker and never treated as directly comparable.

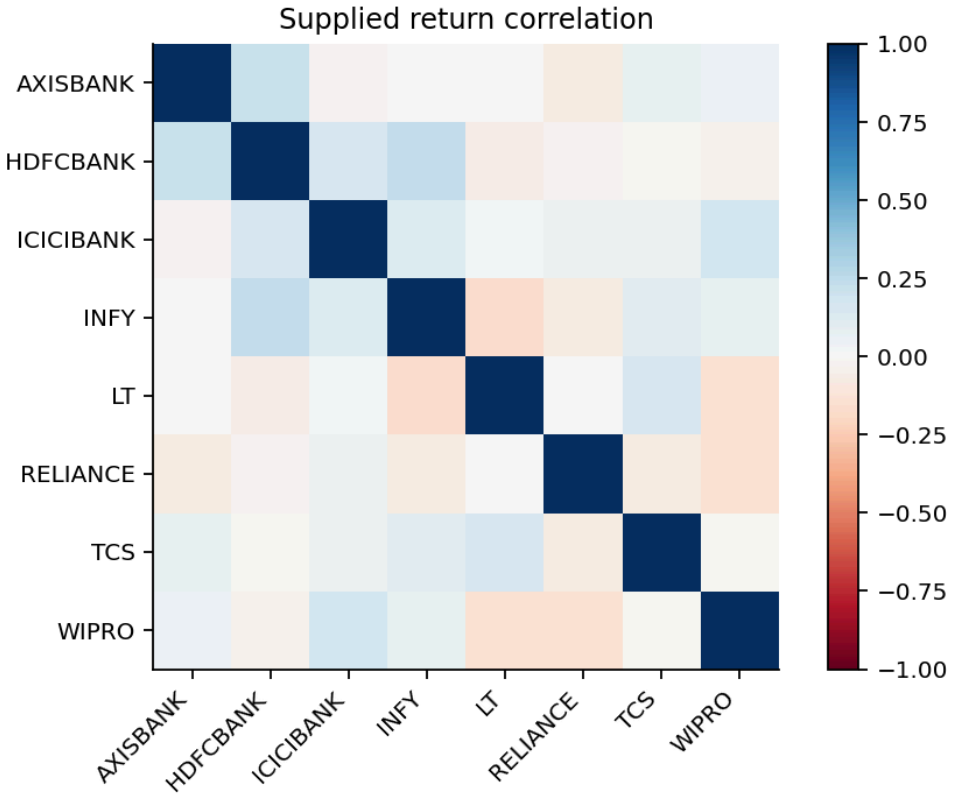


Figure 3: Supplied return correlation heatmap, eight tickers

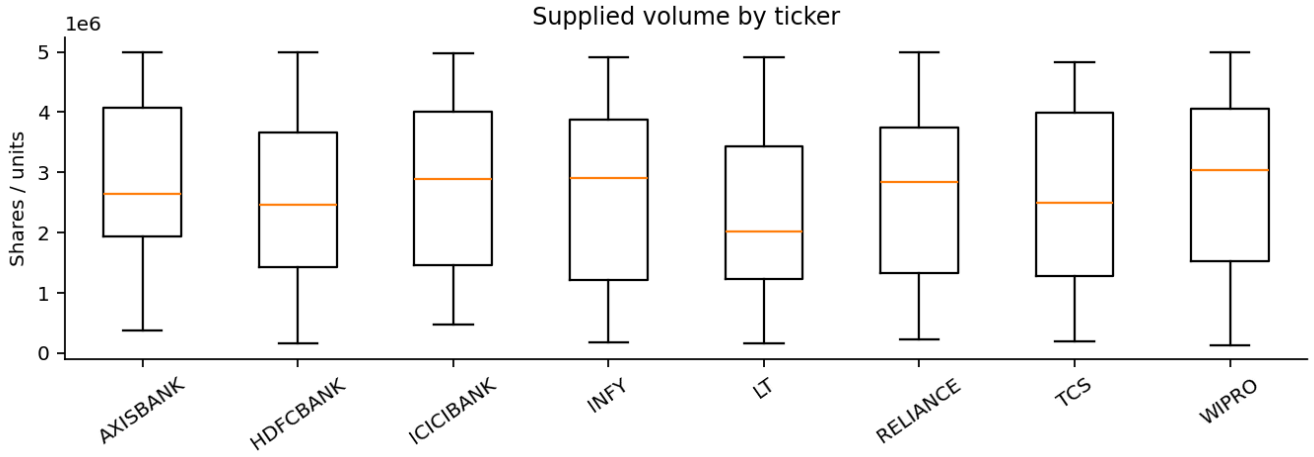


Figure 4: Supplied volume by ticker (box plots)

5. Architecture and Persistence

Streamlit pages call independent Python modules. The data pipeline validates sources and writes prices to SQLite. The portfolio module owns orders and valuation; the risk and signal modules read the same prices; the ML module builds features from past and current observations only. `docs/architecture.md` contains the level-0 data-flow diagram and the block diagram in Mermaid form.

Layer	Responsibility
Streamlit UI	Dashboard, Trade, Risk, Signals, Data Explorer; shared theme and warning
Pipeline	CSV cleaning, yfinance cache, provenance and quality report
SQLite	Stocks, prices, portfolio, transactions, signals and holdings view
Analytics	P&L, risk, technical indicators, backtest and classifier
Deployment	Python 3.11 slim image, non-root user, bind-mounted data, health check

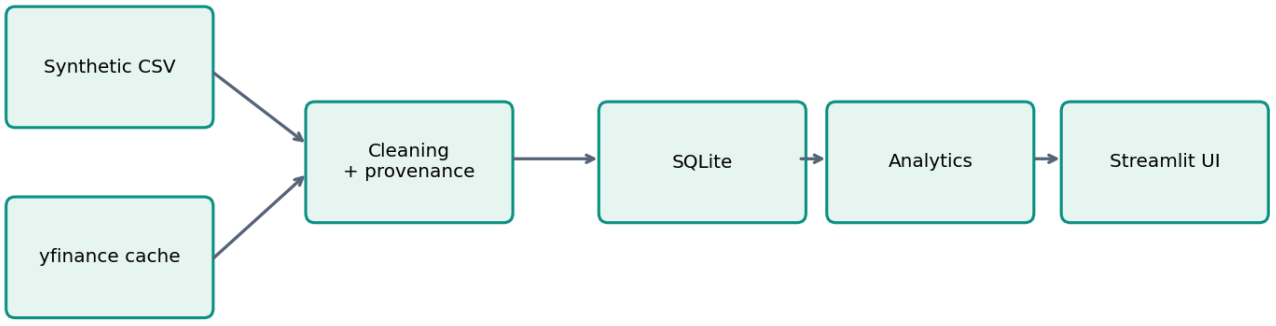


Figure 5: Data flow from synthetic CSV and yfinance cache through cleaning, SQLite and analytics to the Streamlit UI

Transaction integrity

Every order opens an immediate SQLite transaction before cash or shares are read. A buy checks that notional plus fee fits within cash; a sell checks the owned quantity. Order insertion and cash adjustment then commit together or not at all. Foreign keys, primary keys, indexes and CHECK constraints reject inconsistent records, and all SQL parameters are bound rather than interpolated, which also guards against SQL injection.

Reproducibility

A clean clone starts with `docker compose up --build`. On first start the container initialises the database and signals, then launches Streamlit. The data directory is bind-mounted, so trades and cached history survive container replacement. If the network is unavailable, the supplied data still allows a working demonstration.

6. Portfolio Accounting and Risk

The demo portfolio starts with INR 1,000,000 cash.

- **Buy:** notional and brokerage are added to the holding's remaining cost basis.
- **Sell:** quantity is removed at the moving average cost; realised P&L = net proceeds − removed cost.
- **Unrealised P&L:** current market value − remaining basis.
- **Total P&L:** current cash + holdings value − initial funding.
- **Weight:** a holding's market value ÷ total portfolio value.

Verified worked example

The test suite checks this scenario: buy 10 shares at INR 100, sell 4 at INR 120, with INR 20 brokerage on each order.

Quantity	Value
Remaining shares	6
Average cost per share	INR 102
Realised P&L	INR 52
Unrealised P&L (at INR 120)	INR 108
Total P&L	INR 160

Historical risk formulas

- **Annualised volatility** = sample standard deviation of daily returns × √252
- **Sharpe ratio** = (mean daily return − 0.065/252) ÷ daily standard deviation × √252
- **Beta** = covariance(asset, NIFTY 50) ÷ variance(NIFTY 50), on overlapping dates
- **Maximum drawdown** = minimum of (compounded wealth ÷ its previous peak) − 1
- **Historical 95% VaR** = max(0, −5th percentile of daily returns)
- **Correlation** = Pearson's coefficient

Results for TCS (1,235 returns)

Risk measure	Value
Annualised volatility	22.87%

Risk measure	Value
Sharpe ratio (6.5% risk-free)	−0.453
Beta vs NIFTY 50	0.821
Maximum drawdown	−56.46%
Historical one-day VaR (95%)	2.10%

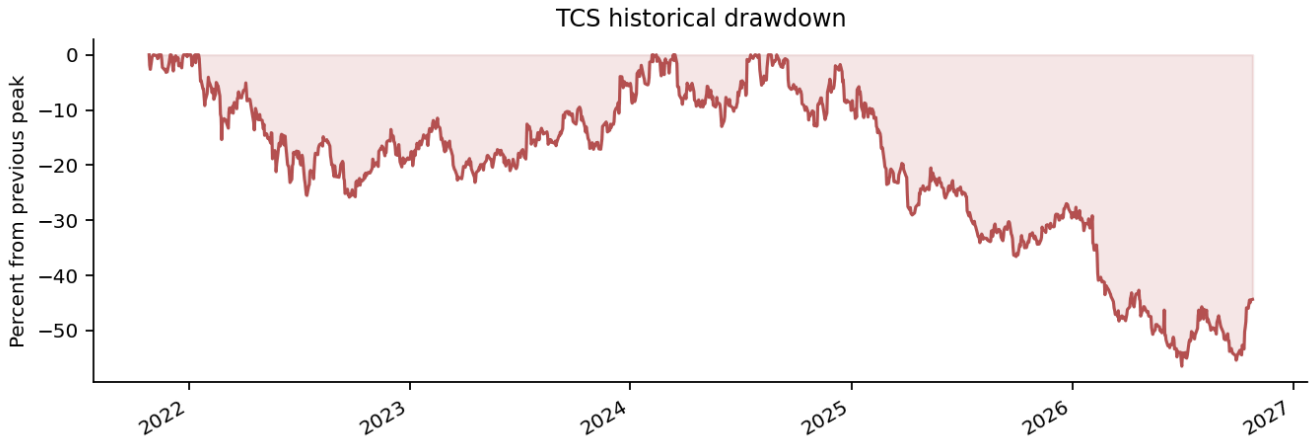


Figure 6: TCS historical drawdown, percent from previous peak

The portfolio risk view applies current shares and cash to historical closes. This is an exposure scenario, not the account's realised value path. All risk figures are historical sample estimates.

7. Signal Simulation and Machine Learning

Technical signals and backtest

The SMA strategy emits a buy or sell signal when the 20-day mean crosses the 50-day mean. RSI(14) uses Wilder-style smoothing and marks oversold readings below 30 and overbought readings above 70. A signal is known only after the day's close, so the backtest trades at the **next open**. The strategy is either long or in cash, buys whole shares only and pays the portfolio's brokerage model. Buy-and-hold buys at the first open for comparison.

TCS SMA backtest measure	Observed result
Strategy total return	−23.94%
Buy-and-hold return	−26.96%
Completed round trips	15

TCS SMA backtest measure	Observed result
Win rate	33.33%
Maximum drawdown	−29.86%

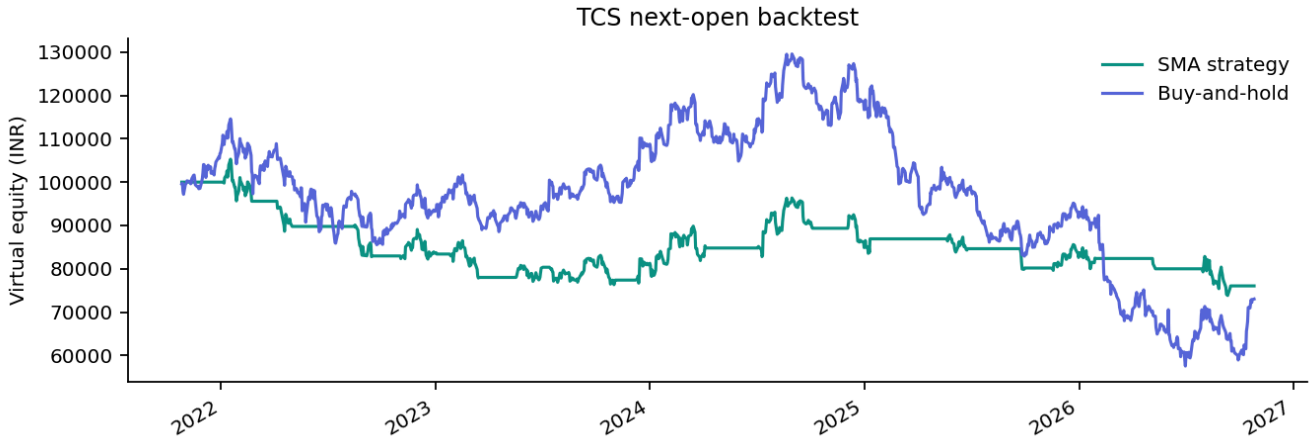


Figure 7: TCS next-open backtest, SMA strategy vs buy-and-hold equity (INR)

Next-day classifier

The classifier predicts next-day direction from features known at the close: price, volume, momentum, range, RSI and rolling volatility. A scaler and logistic regression are fitted on earlier rows only. The last training label is dropped before the test period, so no training label uses a test-period close. The final 20% of rows is evaluated once, with no random shuffling.

TCS holdout measure	Observed result
Train / test rows	946 / 237
Accuracy / majority baseline	52.74% / 53.59%
Precision / recall / F1	0.493 / 0.673 / 0.569

These values come from a single historical sample. The classifier is not used to generate orders, and an accuracy below the majority baseline gives no evidence of a trading advantage.

8. Verification, Limitations and Conclusion

Verification

Focused pytest checks cover cleaning, SQLite constraints, brokerage-aware cost basis, order guards, known beta and drawdown values, next-open execution and chronological ML evaluation. Streamlit's test runner executed the home page and all five feature pages without exceptions, and a live server returned HTTP 200 from `/_stcore/health`. Docker Compose configuration passed validation; running the container image requires an active Docker daemon on the host.

Test data includes a fully known two-day order scenario and an artificial overnight price jump that would expose same-day signal execution. These checks are more informative than tests that merely restate the implementation. The pages were also run against the longer cached history to confirm that risk cards, charts and ML evaluation render with real observations.

Limitations

- Cached historical prices may be revised by the data provider.
- The source CSV is synthetic and too short for annualised inference.
- The backtest covers one selected ticker and models no slippage, taxes, partial fills, liquidity limits or corporate actions beyond the provider's adjusted values.
- The ML result is a one-off chronological holdout, with no uncertainty interval or walk-forward validation.
- Risk estimates depend on the sample period and do not cap future loss.

Conclusion

Portfolio Lab meets its stated objectives: it cleans and documents the supplied data, keeps an auditable virtual ledger with verified accounting, computes standard risk measures against a benchmark, tests signals with realistic next-open execution and evaluates a classifier honestly against a baseline. Its main finding is a disciplined negative one: neither the SMA strategy nor the classifier shows a reliable edge on TCS. Natural extensions are multi-ticker backtests, walk-forward validation and transaction-cost sensitivity analysis.

Ethical compliance

The system uses only public price data and a virtual account. It collects no names, credentials or other personal data, has no live broker connection and shows the required disclaimer on every page. The learner must personally review the final submission,

disclose any tool assistance where required and sign the separate ethical declaration; this report does not claim a personal signature.

References

1. HCLTech Career Shaper, *Project Brief and Dataset Generation Prompt*, supplied in `data/`.
2. Streamlit, *Creating multipage apps using the pages directory*.
3. pandas, *DataFrame.pct_change* documentation.
4. scikit-learn, *TimeSeriesSplit* documentation (cited as the reference for time-ordered validation; the current evaluation uses a single chronological holdout).

> **Simulation only. Not investment advice.** Virtual money; no real brokerage connection.